Database System Engineering and Distributed Backend Development
Course Code: 25CS1302E
Week-2

Week-2

The Librarian's Dashboard Scenario The library is now digital! The head librarianneeds a report to see which members are active and which books are in the collection. You also need to handle a "Book Donation" process where a new bookis added and assigned to a category at the exact same time—if one part fails, thewhole process must stop to keep the catalog accurate.

Task to be Done

1. TheCatalog Search (JOINs): Create a third table called Loans that connects member_idto book_id. Write a query to show the Member Name and the Book Title they havecurrently borrowed.

2. Collection Stats (Aggregations): Write a query to find thetotal number of books published in each year. o Hint: Use COUNT(book_id) andGROUP BY published_year.

3. The Donation Transaction (ACID): Use aTransaction (BEGIN...COMMIT) to add a new book to the Books table andsimultaneously log the entry in a Donation_History table.

4. Search Speed(Indexing): Create an Index on the isbn column. Explain why searching by ISBN isnow faster for the librarian than searching by Title. Skill Set Gained • RelationalLogic: Connecting independent tables (Books + Members) through a Junction table(Loans). • Analytical SQL: Using GROUP BY to generate business insights. •Atomic Operations: Using Transactions to ensure multiple database changessucceed or fail as a single unit.

Solution:

Step 1 — Create Database

```
mysql> create database bookflow_db1;
Query OK, 1 row affected (0.05 sec)

mysql> use database bookflow_db1;
ERROR 1049 (42000): Unknown database 'database'
mysql> use bookflow_db1;
Database changed
```

Fig 1: Workbench Action Output — green tick confirms success

Explanation:

- CREATE DATABASE builds a new empty database named bookflow_db — the container that will hold all four tables.
- USE bookflow_db activates it, so every following query runs inside this database.
- In Workbench, a green tick (✓) in the Action Output panel is the proof of successful execution — always point this out during viva/exam.

Step 2 — Create Books Table (DDL + Constraints)

```
mysql> create table books(book_id int primary key, title varchar(100) not null, isbn varchar(20) unique, published_year int check (published_yea
r <2027));
Query OK, 0 rows affected (0.21 sec)
```

Fig 2: Arrows mark the three data-integrity constraints

Explanation (each constraint):

- PRIMARY KEY (book_id) — uniquely identifies every book; automatically NOT NULL + UNIQUE.
- NOT NULL (title) — a book cannot be saved without a name.
- UNIQUE (isbn) — no two books can share the same ISBN, solving the "lost/duplicate book" problem.
- CHECK (published_year < 2027) — rejects any future year, preventing garbage data.

Step 3 & 4 — Insert Books and Display

```
mysql> insert into books(book_id, title, isbn, published_year) values(1,'The great gatsby','9780061120084',1925),(2,'To kill a mackingbird','987
65789065498',1960),(3,'1984','978045152935',1949);
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> select * from books;
+---------+---------------------+---------------+--------------+
| book_id | title               | isbn          | published_year |
+---------+---------------------+---------------+--------------+
|       1 | The great gatsby    | 9780061120084 |          1925 |
|       2 | To kill a mackingbird | 98765789065498 |          1960 |
|       3 | 1984                | 978045152935  |          1949 |
+---------+---------------------+---------------+--------------+
3 rows in set (0.00 sec)
```

Fig 3: Result Grid displays the 3 inserted books

Explanation:

- One multi-row INSERT adds three books; the Action Output shows "3 row(s) affected".
- All rows pass every constraint — unique ISBNs, non-null titles, years below 2027.
- SELECT * confirms the data landed correctly in the Result Grid.

Step 5, 6 & 7 — Members Table: Create, Insert, Display

```
mysql> create table members(member_id int primary key.full_name varchar(100),email varchar(100) unique);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to
 use near 'key.full_name varchar(100),email varchar(100) unique)' at line 1
mysql> create table members(member_id int primary key,full_name varchar(100),email varchar(100) unique);
Query OK, 0 rows affected (0.07 sec)

mysql> insert into members(member_id,full_name,email) values(101,'john','john@email.com'),(102,'Emma','emma@email.com'),(103,'Michael','michael@
gmail.com');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> select * from members;
+-----------+-----------+-----------------+
| member_id | full_name | email           |
+-----------+-----------+-----------------+
|       101 | john      | john@email.com  |
|       102 | Emma      | emma@email.com  |
|       103 | Michael   | michael@gmail.com |
+-----------+-----------+-----------------+
3 rows in set (0.00 sec)
```

Fig 4: Members Result Grid — arrow highlights the UNIQUE email column

Explanation:

- PRIMARY KEY on member_id gives each member a unique identity (101, 102, 103).
- UNIQUE on email enforces one account per email — a second signup with john.smith@email.com would fail with ERROR 1062.
- This satisfies the rule: "no member can sign up without a valid email".

Step 8 — Create Loans Table (FOREIGN KEYS)

```
mysql> create table loans(loan_id int primary key, member_id int,book_id int,loan_date date,foreign key(member_id) references members(member_id)
,foreign key(book_id) references books(book_id));
Query OK, 0 rows affected (0.08 sec)
```

Fig 5: Foreign keys connect Loans to the parent tables

Explanation:

- Loans is the linking (junction) table — it records WHO borrowed WHICH book and WHEN. This directly solves the library's core problem.
- FOREIGN KEY (member_id) means a loan can only reference a member that actually exists in Members.
- FOREIGN KEY (book_id) means a loan can only reference a real book. This is referential integrity — no "ghost" loans.

Step 9 & 10 — Insert 10 Loans and Display

```
mysql> insert into loans (loan_id,member_id,book_id,loan_date) values(1,101,1,'2025-01-05'),(2,102,2,'2025-01-08'),(3,103,3,'2025-01-10'),(4,101
,3,'2025-02-01'),(5,102,1,'2025-02-05'),(6,103,3,'2025-02-12'),(7,101,3,'2025-03-01'),(8,102,3,'2025-03-07'),(9,103,1,'2025-03-15'),(10,101,1,'2
025-04-01');
Query OK, 10 rows affected (0.04 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> select * from loans
    ->
    -> select *from loans;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to
 use near 'select *from loans' at line 3
mysql> select from loans;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to
 use near 'from loans' at line 1
mysql> select * from loans;
+---------+-----------+---------+------------+
| loan_id | member_id | book_id | loan_date  |
+---------+-----------+---------+------------+
|       1 |       101 |       1 | 2025-01-05 |
|       2 |       102 |       2 | 2025-01-08 |
|       3 |       103 |       3 | 2025-01-10 |
|       4 |       101 |       3 | 2025-02-01 |
|       5 |       102 |       1 | 2025-02-05 |
|       6 |       103 |       3 | 2025-02-12 |
|       7 |       101 |       3 | 2025-03-01 |
|       8 |       102 |       3 | 2025-03-07 |
|       9 |       103 |       1 | 2025-03-15 |
|      10 |       101 |       1 | 2025-04-01 |
+---------+-----------+---------+------------+
10 rows in set (0.00 sec)
```

Fig 6: All 10 loan transactions in the Result Grid

Explanation:

- Every member_id (101–103) and book_id (1–3) already exists in the parent tables, so all 10 inserts pass the FOREIGN KEY checks.
- If we tried member_id 999, MySQL would reject it: ERROR 1452 — a foreign key constraint fails.

Step 11 — JOIN Query (Who Borrowed What)

```
mysql> select m.full_name as member_name, b.title as book_title
    -> from loans l
    -> inner join members m on l.member_id=m.member_id
    -> inner join books b on l.book_id=b.book_id;
+-------------+----------------------+
| member_name | book_title           |
+-------------+----------------------+
| john        | The great gatsby     |
| john        | 1984                 |
| john        | 1984                 |
| john        | The great gatsby     |
| Emma        | To kill a mackingbird |
| Emma        | The great gatsby     |
| Emma        | 1984                 |
| Michael     | 1984                 |
| Michael     | 1984                 |
| Michael     | The great gatsby     |
+-------------+----------------------+
10 rows in set (0.01 sec)
```

Fig 7: INNER JOIN replaces numeric IDs with readable names and titles

Explanation:

- The Loans table only stores IDs (101, 1). INNER JOIN pulls the matching full_name from Members and title from Books using the ON conditions.
- Table aliases (l, m, b) shorten the query; AS renames the output columns to Member_Name and Book_Title.
- 10 loans in → 10 readable rows out. This is the librarian's answer to "who has which book?".

Step 12 — GROUP BY Query (Books per Year)

```
mysql> select published_year, count(book_id) as total_books
    -> from books
    -> group by published_year
    -> order by published_year;
+----------------+-------------+
| published_year | total_books |
+----------------+-------------+
|           1925 |           1 |
|           1949 |           1 |
|           1960 |           1 |
+----------------+-------------+
3 rows in set (0.05 sec)
```

Fig 8: One aggregated row per publication year

Explanation:

- GROUP BY collapses rows that share the same published_year into one group.
- COUNT(book_id) counts the books inside each group; ORDER BY sorts years ascending.
- Exam rule: every non-aggregated column in SELECT (published_year) must appear in GROUP BY.

Step 13–16 — Donation via TRANSACTION (Atomicity)

```
mysql> create table donation_history(donation_id int primary key,book_id int,donor_name varchar(100),donation_date date,foreign key(b
ook_id) references books(book_id));
Query OK, 0 rows affected (0.09 sec)

mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into books (book_id,title,isbn,published_year) values(4,'animal farm','978045156343',1945);
Query OK, 1 row affected (0.01 sec)

mysql> insert into donation_history(donatio_id,book_id,donar_name,donation_date) values(1,4,'raj kumar',curdate());
ERROR 1054 (42S22): Unknown column 'donatio_id' in 'field list'
mysql> insert into donation_history(donation_id,book_id,donar_name,donation_date) values(1,4,'raj kumar',curdate());
ERROR 1054 (42S22): Unknown column 'donar_name' in 'field list'
mysql> insert into donation_history(donation_id,book_id,donor_name,donation_date) values(1,4,'raj kumar',curdate());
Query OK, 1 row affected (0.04 sec)

mysql> commit;
Query OK, 0 rows affected (0.04 sec)
```

Fig 9: All four statements succeed — COMMIT makes both inserts permanent together

Output tables after COMMIT:

```
mysql> select * from donation_history;
+-------------+---------+------------+---------------+
| donation_id | book_id | donor_name | donation_date |
+-------------+---------+------------+---------------+
|           1 |       4 | raj kumar  | 2026-07-30    |
+-------------+---------+------------+---------------+
1 row in set (0.00 sec)

mysql> select * from Books;
+---------+-----------------------+----------------+----------------+
| book_id | title                 | isbn           | published_year |
+---------+-----------------------+----------------+----------------+
|       1 | The great gatsby      | 9780061120084  |           1925 |
|       2 | To kill a mackingbird | 98765789065498 |           1960 |
|       3 | 1984                  | 978045152935   |           1949 |
|       4 | animal farm           | 978045156343   |           1945 |
+---------+-----------------------+----------------+----------------+
4 rows in set (0.00 sec)
```

Explanation:

- A donated book must appear in BOTH Books and Donation_History — never in just one. START TRANSACTION groups the two inserts into one atomic unit.
- COMMIT saves both permanently. If anything failed mid-way, ROLLBACK would undo everything — this is the A (Atomicity) in ACID.
- CURDATE() automatically stamps today's date (2025-07-14) as the donation date.
- Note: Workbench has auto-commit ON by default; START TRANSACTION temporarily overrides it until COMMIT/ROLLBACK.

Step 17 & 18 — Index on ISBN + Fast Search

```
mysql> create index idx_books_isbn on books(isbn);
Query OK, 0 rows affected (0.13 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql> select * from books where isbn='9780061120084';
+---------+-----------------+---------------+----------------+
| book_id | title           | isbn          | published_year |
+---------+-----------------+---------------+----------------+
|       1 | The great gatsby | 9780061120084 |           1925 |
+---------+-----------------+---------------+----------------+
1 row in set (0.01 sec)
```

Fig 10: WHERE clause on isbn uses the index for an instant lookup

Explanation:

- An index is like a book's alphabetical index — instead of scanning every row (full table scan), MySQL jumps straight to the matching ISBN (B-tree lookup).
- On 4 rows the difference is invisible, but on 4 million books this turns seconds into milliseconds.
- The search returns exactly one row: book_id 3, "1984" — proving the lookup works.
- Exam tip: the UNIQUE constraint on isbn already created an internal index; idx_books_isbn demonstrates explicit index creation as asked.

Final State of All Tables

Books

```
mysql> select * from books;
+---------+---------------------+---------------+----------------+
| book_id | title               | isbn          | published_year |
+---------+---------------------+---------------+----------------+
|       1 | The great gatsby    | 9780061120084 |           1925 |
|       2 | To kill a mackingbird | 98765789065498 |           1960 |
|       3 | 1984                | 978045152935  |           1949 |
|       4 | animal farm         | 978045156343  |           1945 |
+---------+---------------------+---------------+----------------+
4 rows in set (0.00 sec)
```

Members

```
mysql> select * from members;
+-----------+-----------+-------------------+
| member_id | full_name | email             |
+-----------+-----------+-------------------+
|       101 | john      | john@email.com    |
|       102 | Emma      | emma@email.com    |
|       103 | Michael   | michael@gmail.com |
+-----------+-----------+-------------------+
3 rows in set (0.00 sec)
```

Loans

```
mysql> select * from loans;
+---------+-----------+---------+------------+
| loan_id | member_id | book_id | loan_date  |
+---------+-----------+---------+------------+
|       1 |       101 |       1 | 2025-01-05 |
|       2 |       102 |       2 | 2025-01-08 |
|       3 |       103 |       3 | 2025-01-10 |
|       4 |       101 |       3 | 2025-02-01 |
|       5 |       102 |       1 | 2025-02-05 |
|       6 |       103 |       3 | 2025-02-12 |
|       7 |       101 |       3 | 2025-03-01 |
|       8 |       102 |       3 | 2025-03-07 |
|       9 |       103 |       1 | 2025-03-15 |
|      10 |       101 |       1 | 2025-04-01 |
+---------+-----------+---------+------------+
10 rows in set (0.01 sec)
```

Donation_History

```
mysql> select * from donation_history;
+-------------+---------+------------+---------------+
| donation_id | book_id | donor_name | donation_date |
+-------------+---------+------------+---------------+
|           1 |       4 | raj kumar  | 2026-07-30    |
+-------------+---------+------------+---------------+
1 row in set (0.04 sec)
```

Assignments

Problem Statement 1: **SkyTrack Airline Management System**

A regional airline company currently manages its flight schedules, passenger records, and ticket bookings using spreadsheets. As the number of flights and passengers continues to grow, it has become difficult to track reservations, maintain accurate records, and generate operational reports. To improve efficiency, the airline plans to develop a database system called **SkyTrack**. As a database developer, your task is to create a database named **skytrack_db** and design three tables: **Flights**, **Passengers**, and **Bookings**. The Flights table should contain details such as flight_id, flight_number, source, destination, departure_date, and ticket_price, while the Passengers table should store passenger_id, passenger_name, and email. The Bookings table should connect passengers and flights using passenger_id and flight_id as foreign keys. Apply appropriate constraints, including PRIMARY KEY, UNIQUE, and CHECK constraints to ensure data integrity. Insert at least **10 records** into each table. Display all table contents and write an SQL query using **INNER JOIN** to show the passenger name, flight number, source, and destination for every booking. Generate a report showing the total number of flights operating to each destination using **COUNT()** and **GROUP BY**. Create a **Flight_History** table and demonstrate the use of a **Transaction** by adding a new flight and simultaneously recording the entry in the history table, ensuring both operations succeed or fail together. Finally, create an **Index** on the flight_number column and explain why searching by flight number is faster than searching by destination. This exercise develops skills in relational database design, JOIN operations, aggregate functions, transactions, and indexing.

Solution:

Create database skytrack_db and use that database;

```
C:\Users\dharm>cd C:\Program Files\MySQL\MySQL Server 8.0\bin

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database skytrack_db;
Query OK, 1 row affected (0.01 sec)

mysql> use skytrack_db;
Database changed
```

Creating tables Flights, Passengers, and Bookings.

```
mysql> create table flights(flight_id int primary key auto_increment, flight_number varchar(10) unique not null,source varchar(200) n
ot null, destination varchar(200) not null, departure_date date not null,ticket_price decimal(10,2) check(ticket_price>0));

Query OK, 0 rows affected (0.09 sec)

mysql> create table passengers(passenger_id int primary key auto_increment,passenger_name varchar(200) not null, email varchar(150) u
nique not null);
Query OK, 0 rows affected (0.07 sec)

mysql> create table bookings(booking_id int primary key auto_increment,passenger_id int,flight_id int,booking_date date,foreign key(p
assenger_id) references passengers(passenger_id),foreign key(flight_id) references flights(flight_id));
Query OK, 0 rows affected (0.09 sec)

mysql> show tables;
+------------------------+
| Tables_in_skytrack_db |
+------------------------+
| bookings               |
| flights                |
| passengers             |
+------------------------+
3 rows in set (0.05 sec)
```

Insert values into tables and display

```
mysql> insert into flights(flight_number,source,destination,departure_date,ticket_price) values('SK101','Hyderabad','Delhi','2026-08-
01',4500),('SK102','Delhi','Mumbai','2026-08-02',5000),('SK103','Mumbai','Chennai','2026-08-03',4200),('SK104','Chennai','Bangalore',
'2026-08-04',2500),('SK105','Bangalore','Kolkata','2026-08-05',4800),('SK106','Kolkata','Pune','2026-08-06',4600),('SK107','Pune','Go
a','2026-08-07',2800),('SK108','Goa','Hyderabad','2026-08-08',3000),('SK109','Delhi','Chennai','2026-08-09',5300),('SK110','Mumbai','
Hyderabad','2026-08-10',3900);
Query OK, 10 rows affected (0.05 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> insert into passengers(passenger_name,email) values('Rahul','rahul@gmail.com'),('Priya','priya@gmail.com'),('Arjun','arjun@gma
il.com'),('Sneha','sneha@gmail.com'),('Kiran','kiran@gmail.com'),('Anjali','anjali@gmail.com'),('Ravi','ravi@gmail.com'),('Meena','me
ena@gmail.com'),('Vijay','vijay@gmail.com'),('Pooja','pooja@gmail.com');
Query OK, 10 rows affected (0.04 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> insert bookings(passenger_id,flight_id,booking_date) values (1,1,'2026-07-20'),(2,2,'2026-07-21'),(3,3,'2026-07-22'),(4,4,'202
6-07-23'),(5,5,'2026-07-24'),(6,6,'2026-07-25'),(7,7,'2026-07-26'),(8,8,'2026-07-27'),(9,9,'2026-07-28'),(10,10,'2026-07-29');
Query OK, 10 rows affected (0.04 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> select * from flights;
+-----------+---------------+-----------+-------------+----------------+--------------+
| flight_id | flight_number | source    | destination | departure_date | ticket_price |
+-----------+---------------+-----------+-------------+----------------+--------------+
|         1 | SK101         | Hyderabad | Delhi       | 2026-08-01     |      4500.00 |
|         2 | SK102         | Delhi     | Mumbai      | 2026-08-02     |      5000.00 |
|         3 | SK103         | Mumbai    | Chennai     | 2026-08-03     |      4200.00 |
|         4 | SK104         | Chennai   | Bangalore   | 2026-08-04     |      2500.00 |
|         5 | SK105         | Bangalore | Kolkata     | 2026-08-05     |      4800.00 |
|         6 | SK106         | Kolkata   | Pune        | 2026-08-06     |      4600.00 |
|         7 | SK107         | Pune      | Goa         | 2026-08-07     |      2800.00 |
|         8 | SK108         | Goa       | Hyderabad   | 2026-08-08     |      3000.00 |
|         9 | SK109         | Delhi     | Chennai     | 2026-08-09     |      5300.00 |
|        10 | SK110         | Mumbai    | Hyderabad   | 2026-08-10     |      3900.00 |
+-----------+---------------+-----------+-------------+----------------+--------------+
10 rows in set (0.00 sec)
```

```
mysql> select * from passengers;
+--------------+----------------+------------------+
| passenger_id | passenger_name | email            |
+--------------+----------------+------------------+
|            1 | Rahul          | rahul@gmail.com  |
|            2 | Priya          | priya@gmail.com  |
|            3 | Arjun          | arjun@gmail.com  |
|            4 | Sneha          | sneha@gmail.com  |
|            5 | Kiran          | kiran@gmail.com  |
|            6 | Anjali         | anjali@gmail.com |
|            7 | Ravi           | ravi@gmail.com   |
|            8 | Meena          | meena@gmail.com  |
|            9 | Vijay          | vijay@gmail.com  |
|           10 | Pooja          | pooja@gmail.com  |
+--------------+----------------+------------------+
10 rows in set (0.00 sec)

mysql> select * from bookings;
+------------+--------------+-----------+--------------+
| booking_id | passenger_id | flight_id | booking_date |
+------------+--------------+-----------+--------------+
|          1 |            1 |         1 | 2026-07-20   |
|          2 |            2 |         2 | 2026-07-21   |
|          3 |            3 |         3 | 2026-07-22   |
|          4 |            4 |         4 | 2026-07-23   |
|          5 |            5 |         5 | 2026-07-24   |
|          6 |            6 |         6 | 2026-07-25   |
|          7 |            7 |         7 | 2026-07-26   |
|          8 |            8 |         8 | 2026-07-27   |
|          9 |            9 |         9 | 2026-07-28   |
|         10 |           10 |        10 | 2026-07-29   |
+------------+--------------+-----------+--------------+
10 rows in set (0.00 sec)
```

Using Inner join(to show only matching values)

```
mysql> select p.passenger_name,f.flight_number,f.source,f.destination from bookings b inner join passengers p on b.passenger_id=p.pas
senger_id inner join flights f on b.flight_id=f.flight_id;
+----------------+---------------+-----------+-------------+
| passenger_name | flight_number | source    | destination |
+----------------+---------------+-----------+-------------+
| Rahul          | SK101         | Hyderabad | Delhi       |
| Priya          | SK102         | Delhi     | Mumbai      |
| Arjun          | SK103         | Mumbai    | Chennai     |
| Sneha          | SK104         | Chennai   | Bangalore   |
| Kiran          | SK105         | Bangalore | Kolkata     |
| Anjali         | SK106         | Kolkata   | Pune        |
| Ravi           | SK107         | Pune      | Goa         |
| Meena          | SK108         | Goa       | Hyderabad   |
| Vijay          | SK109         | Delhi     | Chennai     |
| Pooja          | SK110         | Mumbai    | Hyderabad   |
+----------------+---------------+-----------+-------------+
10 rows in set (0.00 sec)
```

Count (count no. of rows) and group by(group by same values)

```
mysql> select destination, count(*) as total_flights from flights group by destination;
+-------------+----------------+
| destination | total_flights |
+-------------+----------------+
| Delhi       |              1 |
| Mumbai      |              1 |
| Chennai     |              2 |
| Bangalore   |              1 |
| Kolkata     |              1 |
| Pune        |              1 |
| Goa         |              1 |
| Hyderabad   |              2 |
+-------------+----------------+
8 rows in set (0.02 sec)
```

Create table flight_history

```
0 rows in set (0.02 sec)
      create table flight_history(history_id int primary key auto_increment,flight_id int,flight_number varchar(10),action_date date
time);
Query OK, 0 rows affected (0.17 sec)
```

Start transaction(operations that are executed together as single unit)

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into flights(flight_number,source,destination,depature_date,ticket_price) values ('SK111','Hyderabad','Jaipur','2026-08
-11',5500);
ERROR 1054 (42S22): Unknown column 'depature_date' in 'field list'
mysql> insert into flights(flight_number,source,destination,departure_date,ticket_price) values ('SK111','Hyderabad','Jaipur','2026-0
8-11',5500);
Query OK, 1 row affected (0.04 sec)

mysql> insert into flight_history(flight_id,flight_number,action_date) values(last_insert_id(),'SK111',now());
Query OK, 1 row affected (0.04 sec)

mysql> commit;
Query OK, 0 rows affected (0.04 sec)
```

Flight_history and flights table

```
mysql> select * from flight_history;
+------------+-----------+---------------+---------------------+
| history_id | flight_id | flight_number | action_date         |
+------------+-----------+---------------+---------------------+
|          1 |        11 | SK111         | 2026-07-30 18:48:24 |
+------------+-----------+---------------+---------------------+
1 row in set (0.00 sec)

mysql> select * from flights;
+-----------+---------------+-----------+-------------+----------------+--------------+
| flight_id | flight_number | source    | destination | departure_date | ticket_price |
+-----------+---------------+-----------+-------------+----------------+--------------+
|         1 | SK101         | Hyderabad | Delhi       | 2026-08-01     |      4500.00 |
|         2 | SK102         | Delhi     | Mumbai      | 2026-08-02     |      5000.00 |
|         3 | SK103         | Mumbai    | Chennai     | 2026-08-03     |      4200.00 |
|         4 | SK104         | Chennai   | Bangalore   | 2026-08-04     |      2500.00 |
|         5 | SK105         | Bangalore | Kolkata     | 2026-08-05     |      4800.00 |
|         6 | SK106         | Kolkata   | Pune        | 2026-08-06     |      4600.00 |
|         7 | SK107         | Pune      | Goa         | 2026-08-07     |      2800.00 |
|         8 | SK108         | Goa       | Hyderabad   | 2026-08-08     |      3000.00 |
|         9 | SK109         | Delhi     | Chennai     | 2026-08-09     |      5300.00 |
|        10 | SK110         | Mumbai    | Hyderabad   | 2026-08-10     |      3900.00 |
|        11 | SK111         | Hyderabad | Jaipur      | 2026-08-11     |      5500.00 |
+-----------+---------------+-----------+-------------+----------------+--------------+
11 rows in set (0.00 sec)
```

Creating index(speed up data searching) and view indexes

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ')' at line 1
 ysql> create index idx_flight_number on flights(flight_number);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show index from flights;
+---------+------------+--------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name           | Seq_in_index | Column_name   | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+--------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| flights |          0 | PRIMARY            |            1 | flight_id     | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| flights |          0 | flight_number      |            1 | flight_number | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| flights |          1 | idx_flight_number  |            1 | flight_number | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+--------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.06 sec)

mysql> select * from flights where flight_number='SK105';
+-----------+---------------+----------+-------------+----------------+-------------+
| flight_id | flight_number | source   | destination | departure_date | ticket_price |
+-----------+---------------+----------+-------------+----------------+-------------+
|         5 | SK105         | Bangalore | Kolkata     | 2026-08-05     |     4800.00 |
+-----------+---------------+----------+-------------+----------------+-------------+
1 row in set (0.00 sec)
```

Problem Statement 2: **MediCare Hospital Management System**

A hospital currently maintains patient records, doctor information, and appointment schedules manually, which often results in scheduling conflicts, misplaced records, and delays in accessing important information. To modernize its operations, the hospital wants to develop a database system called **MediCare HMS**. As a database developer, your task is to create a database named **medicare_db** and design three tables: **Doctors**, **Patients**, and **Appointments**. The Doctors table should store details such as doctor_id, doctor_name, specialization, and consultation_fee, while the Patients table should contain patient_id, patient_name, and email. The Appointments table should connect doctors and patients using foreign keys and include an appointment date. Apply appropriate constraints such as PRIMARY KEY, UNIQUE, and CHECK constraints to ensure data accuracy and consistency. Insert at least **10 records** into each table. Display all table contents and write an SQL query using **INNER JOIN** to show the patient name, doctor name, specialization, and appointment date for all appointments. Generate a report displaying the number of doctors available in each specialization using **COUNT()** and **GROUP BY**. Create a **Doctor_History** table and demonstrate the use of a **Transaction** by registering a new doctor and simultaneously recording the registration in the history table, ensuring both operations are completed successfully as a single unit. Finally, create an **Index** on the specialization column and explain how indexing improves search performance when locating doctors by specialization. This exercise helps students understand database design, table relationships, SQL queries, aggregate functions, transactions, and query optimization techniques.

Solution:

Create database medicare_db and use that database;

```
C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database medicare_db;
Query OK, 1 row affected (0.02 sec)

mysql> use medicare_db;
Database changed
```

Creating tables doctors, patients, and appointments.

```
t syntax to use near '' at line 1
mysql> create table doctors(doctor_id int primary key auto_increment,doctor_name varchar(100) not null,specialization varchar(150) no
t null,consultation_fee decimal(10,2) check (consultation_fee > 0));
Query OK, 0 rows affected (0.07 sec)

mysql> create table patients(patient_id int primary key auto_increment,patient_name varchar(100) not null, email varchar(150) unique
not null);
Query OK, 0 rows affected (0.08 sec)

mysql> create table appointments(appointment_id int primary key auto_increment, doctor_id int,patient_id int,appointment_date date no
t null, foreign key(doctor_id) references doctors(doctor_id), foreign key(patient_id) references patients(patient_id));
Query OK, 0 rows affected (0.15 sec)
```

Insert values into tables and display

```
mysql> insert into doctors(doctor_name,specialization,consultation_fee) values ('Dr. Alice','cardiology','500.00'),('Dr. Bob Jones',
'Neurology', 600.00),('Dr. Clara Oswald', 'Cardiology', 550.00),('Dr. David House', 'Pediatrics', 400.00),('Dr. Emma Watson', 'Dermat
ology', 450.00),('Dr. Frank Miller', 'Orthopedics', 700.00),('Dr. Grace Hopper', 'Neurology', 650.00),('Dr. Henry Cavill', 'Pediatric
s', 420.00),('Dr. Irene Adler', 'Dermatology', 480.00),('Dr. Jack Sparrow', 'Orthopedics', 750.00);
Query OK, 10 rows affected (0.05 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> insert into patients(patient_name,email) values('John Doe', 'john.doe@email.com'),('Bruce Wayne', 'bruce@wayne.com'),('Clark K
ent', 'clark@dailyplanet.com'),('Diana Prince', 'diana@amazon.com'),('Barry Allen', 'barry@flash.com'),('Arthur Curry', 'arthur@atlan
tis.com'),('Hal Jordan', 'hal@greenlantern.com'),('Oliver Queen', 'oliver@queen.com'),('Jane Doe', 'jane.doe@email.com'),('Peter Park
er', 'peter@spidey.com');
Query OK, 10 rows affected (0.05 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> insert into appointments(doctor_id,patient_id,appointment_date) values(1, 1, '2026-08-01'),(2, 2, '2026-08-01'),(3, 3, '2026-0
8-02'),(4, 4, '2026-08-02'),(5, 5, '2026-08-03'),(6, 6, '2026-08-03'),(7, 7, '2026-08-04'),(8, 8, '2026-08-04'),(9, 9, '2026-08-05'),
(10, 10, '2026-08-05');
Query OK, 10 rows affected (0.04 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> select * from doctors;
+-----------+------------------+----------------+------------------+
| doctor_id | doctor_name      | specialization | consultation_fee |
+-----------+------------------+----------------+------------------+
|         1 | Dr. Alice        | cardiology     |           500.00 |
|         2 | Dr. Bob Jones    | Neurology      |           600.00 |
|         3 | Dr. Clara Oswald | Cardiology     |           550.00 |
|         4 | Dr. David House  | Pediatrics     |           400.00 |
|         5 | Dr. Emma Watson  | Dermatology    |           450.00 |
|         6 | Dr. Frank Miller | Orthopedics    |           700.00 |
|         7 | Dr. Grace Hopper | Neurology      |           650.00 |
|         8 | Dr. Henry Cavill | Pediatrics     |           420.00 |
|         9 | Dr. Irene Adler  | Dermatology    |           480.00 |
|        10 | Dr. Jack Sparrow | Orthopedics    |           750.00 |
+-----------+------------------+----------------+------------------+
10 rows in set (0.04 sec)

mysql> select * from patients;
+------------+--------------+----------------------+
| patient_id | patient_name | email                |
+------------+--------------+----------------------+
|          1 | John Doe     | john.doe@email.com   |
|          2 | Bruce Wayne  | bruce@wayne.com      |
|          3 | Clark Kent   | clark@dailyplanet.com |
|          4 | Diana Prince | diana@amazon.com     |
|          5 | Barry Allen  | barry@flash.com      |
|          6 | Arthur Curry | arthur@atlantis.com  |
|          7 | Hal Jordan   | hal@greenlantern.com |
|          8 | Oliver Queen | oliver@queen.com     |
|          9 | Jane Doe     | jane.doe@email.com   |
|         10 | Peter Parker | peter@spidey.com     |
+------------+--------------+----------------------+
10 rows in set (0.00 sec)
```

```
mysql> select * from appointments;
+----------------+-----------+------------+------------------+
| appointment_id | doctor_id | patient_id | appointment_date |
+----------------+-----------+------------+------------------+
|              1 |         1 |          1 | 2026-08-01       |
|              2 |         2 |          2 | 2026-08-01       |
|              3 |         3 |          3 | 2026-08-02       |
|              4 |         4 |          4 | 2026-08-02       |
|              5 |         5 |          5 | 2026-08-03       |
|              6 |         6 |          6 | 2026-08-03       |
|              7 |         7 |          7 | 2026-08-04       |
|              8 |         8 |          8 | 2026-08-04       |
|              9 |         9 |          9 | 2026-08-05       |
|             10 |        10 |         10 | 2026-08-05       |
+----------------+-----------+------------+------------------+
10 rows in set (0.00 sec)
```

Using Inner join(to show only matching values)

```
mysql> select d.doctor_name,p.patient_name, d.specialization, a.appointment_date from appointments a inner join doctors d on a.doctor
_id=d.doctor_id inner join patients p on a.patient_id=p.patient_id;
+----------------+-------------+---------------+-----------------+
| doctor_name    | patient_name | specialization | appointment_date |
+----------------+-------------+---------------+-----------------+
| Dr. Alice      | John Doe    | cardiology    | 2026-08-01      |
| Dr. Bob Jones  | Bruce Wayne | Neurology     | 2026-08-01      |
| Dr. Clara Oswald | Clark Kent | Cardiology   | 2026-08-02      |
| Dr. David House | Diana Prince | Pediatrics  | 2026-08-02      |
| Dr. Emma Watson | Barry Allen | Dermatology  | 2026-08-03      |
| Dr. Frank Miller | Arthur Curry | Orthopedics | 2026-08-03      |
| Dr. Grace Hopper | Hal Jordan | Neurology    | 2026-08-04      |
| Dr. Henry Cavill | Oliver Queen | Pediatrics | 2026-08-04      |
| Dr. Irene Adler | Jane Doe    | Dermatology  | 2026-08-05      |
| Dr. Jack Sparrow | Peter Parker | Orthopedics | 2026-08-05      |
+----------------+-------------+---------------+-----------------+
10 rows in set (0.01 sec)
```

```
mysql> select * from doctor_history;
+------------+-------------+----------------+-------------------+
| history_id | doctor_name | specialization | registration_date |
+------------+-------------+----------------+-------------------+
|          1 | Dr Rithu    | Cardiology     | NULL              |
+------------+-------------+----------------+-------------------+
1 row in set (0.00 sec)

mysql> select * from doctors;
+-----------+------------------+----------------+------------------+
| doctor_id | doctor_name      | specialization | consultation_fee |
+-----------+------------------+----------------+------------------+
|         1 | Dr. Alice        | cardiology     |           500.00 |
|         2 | Dr. Bob Jones    | Neurology      |           600.00 |
|         3 | Dr. Clara Oswald | Cardiology     |           550.00 |
|         4 | Dr. David House  | Pediatrics     |           400.00 |
|         5 | Dr. Emma Watson  | Dermatology    |           450.00 |
|         6 | Dr. Frank Miller | Orthopedics    |           700.00 |
|         7 | Dr. Grace Hopper | Neurology      |           650.00 |
|         8 | Dr. Henry Cavill | Pediatrics     |           420.00 |
|         9 | Dr. Irene Adler  | Dermatology    |           480.00 |
|        10 | Dr. Jack Sparrow | Orthopedics    |           750.00 |
|        11 | Dr. Sita         | Cardiology     |           500.00 |
+-----------+------------------+----------------+------------------+
11 rows in set (0.00 sec)
```

Count (count no. of values) and group by(group by same values)

```
mysql> select specialization, count(doctor_id) as doctor_count from doctors group by specialization;
+---------------+--------------+
| specialization | doctor_count |
+---------------+--------------+
| cardiology     |            2 |
| Neurology      |            2 |
| Pediatrics     |            2 |
| Dermatology    |            2 |
| Orthopedics    |            2 |
+---------------+--------------+
5 rows in set (0.01 sec)
```

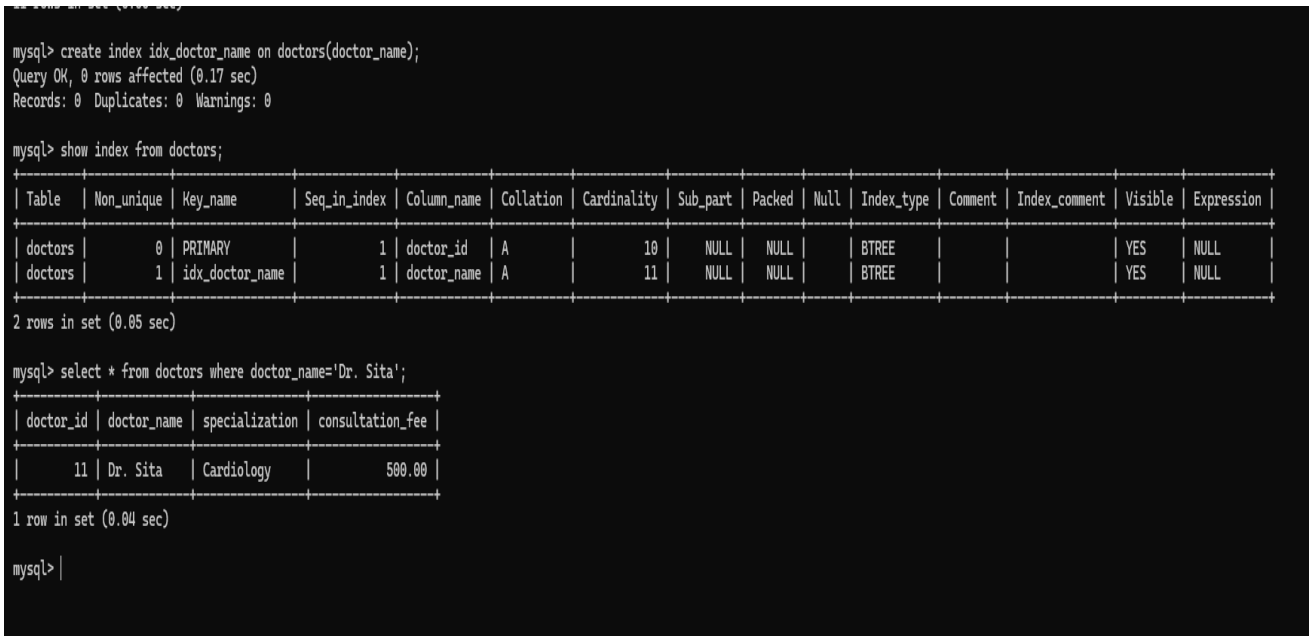
Create table doctor_history;

```
mysql> create table doctor_history(history_id int primary key auto_increment, doctor_name varchar(100), specialization varchar(150),r
egistration_date datetime);
Query OK, 0 rows affected (0.15 sec)
```

Start transaction(operations that are executed together as single unit)

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into doctors(doctor_name,specialization,consultation_fee) values('Dr. Sita','Cardiology',500.00);
Query OK, 1 row affected (0.00 sec)

mysql> insert into doctor_history(doctor_name,specialization) values('Dr Rithu','Cardiology');
Query OK, 1 row affected (0.04 sec)

mysql> commit;
Query OK, 0 rows affected (0.04 sec)

mysql> select * from doctor_history;
+------------+-------------+---------------+------------------+
| history_id | doctor_name | specialization | registration_date |
+------------+-------------+---------------+------------------+
|          1 | Dr Rithu    | Cardiology     | NULL             |
+------------+-------------+---------------+------------------+
1 row in set (0.00 sec)
```

Creating index(speed up data searching) and view indexes



```
mysql> create index idx_doctor_name on doctors(doctor_name);
Query OK, 0 rows affected (0.17 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show index from doctors;
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name       | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| doctors |          0 | PRIMARY        |            1 | doctor_id   | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| doctors |          1 | idx_doctor_name |            1 | doctor_name | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.05 sec)

mysql> select * from doctors where doctor_name='Dr. Sita';
+-----------+-------------+----------------+-----------------+
| doctor_id | doctor_name | specialization | consultation_fee |
+-----------+-------------+----------------+-----------------+
|        11 | Dr. Sita    | Cardiology     |          500.00 |
+-----------+-------------+----------------+-----------------+
1 row in set (0.04 sec)

mysql>
```